13/03

```
struct DOB
{
        int day;
        int mon;
        int year;
};
```

12 bytes

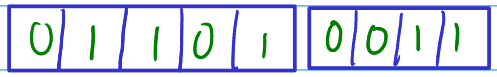
we know that day cannot exceed 31 value
month cannot exceed 12 value
here memory getting wasted, because to value 31 we dont require 32 bits, same for value 12 also.
To make use of memory efficently we go structure bit fields

syntax to declare bit fields for structures

datatype struct member : width

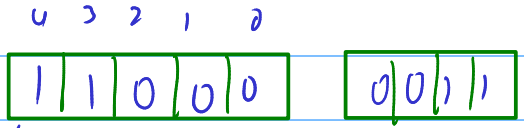
ex :

```
struct DOB
{
        short int day : 5;
        short int month : 4;
        int year;
};
```




```
struct DOB today={13,3,2024};
struct DOB v={24,3,2024};
```

-8-03-2024



24

Signed bit

```
struct DOB
{       unsigned short int day:5;
        unisgned short int mon : 4;
        unsigned short int year;
}v={24,03,2024}
```

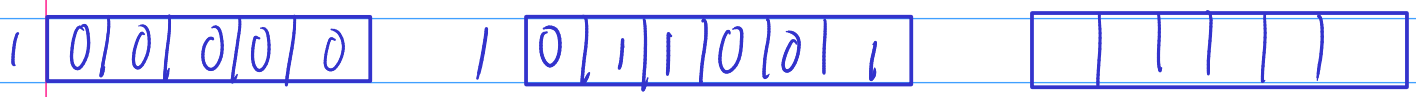
00 111
1
01000

-> 24-03-2024

```
struct Time
{
        unsigned short int hr : 5;
        unsigned short int min : 6;
        unsigned short int sec : 6;
}v={32,89,56};
```

1
64
16
80

0 25



100000 1011001

When we request for bitfields, compiler may reserve first bits or lastbits or inbetween bits its purely compiler choice , we cannot predict it

Points to be noted :

1. bit fields are applicable only to structures and unions datatypes
2. we cannot access or apply address operator for bitfield members.
3. We cannot initialize the bitfield members at runtime as we cannot access their address.
4. We cannot apply bitfields to array members

ex :

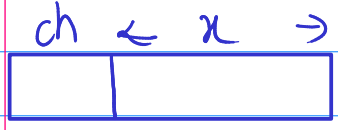
```
struct St
{       int x;
        int a[5] : 10;
};
```

X Invalid compilation error

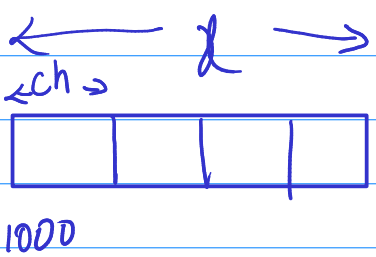
Union

Union is user defined datatype same like structue.
Only difference comes with size.
In structures size is sum of the sizes of the members of the structure,
where as in union it takes the highest member size

```
struct ST
{
        char ch;
        int x;
};
```



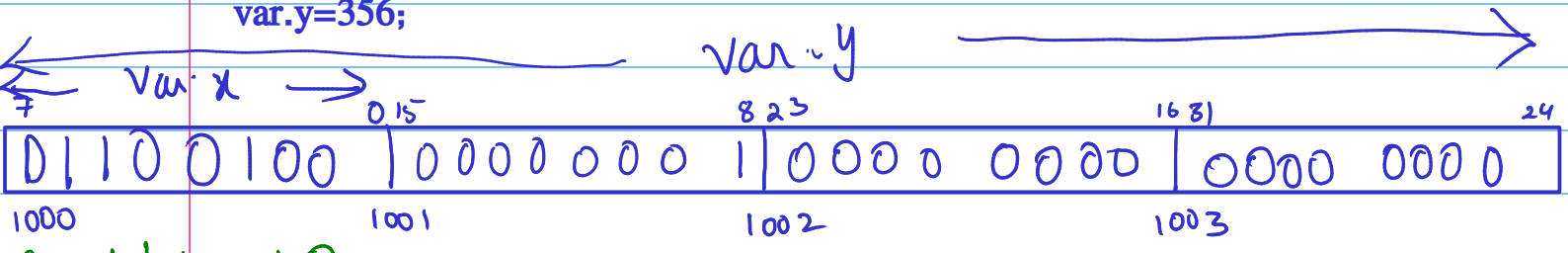

```
union Ut
{
        char ch;
        int x;
};
```




```
union Ut
{
        char x;
        int y;
};

union Ut var;
var.y=356;
```

356 → 256+100



0 1 1 1 0 1 0 0

96
16
6
120

var.y = 356
var.x = 'd'
var.x = 'B' ; // 122
var.x → 'B'
var.y → 356

Unions are used

1. when we want same data in different types
2. checking for Endianess of the system
3. printing of binary equivalent of a float or double